

03 Normal Forms

1. Introduction to Normalization

- Normalization is the process of structuring a relational database to minimize redundancy and dependency.
- Ensures that data is efficiently stored and anomalies (insertion, deletion, and update issues) are avoided.
- Follows a step-by-step process to convert an unnormalized relational model into a series of normalized forms.

2. Issues in Unnormalized Relational Models

2.1 Insertion Anomalies

- Adding new data may require redundant information.
- Example: In a **SUPPLIES** table, adding a new product without a supplier may not be possible if the primary key includes both **SUPNR** (supplier number) and **PRODNR** (product number).

2.2 Deletion Anomalies

- Deleting data may remove important linked information.
- Example: If a supplier is deleted, all associated products could be lost.

2.3 Update Anomalies

- Updating a value may require multiple changes across records.
- Example: Changing a supplier's address in a **SUPPLIES** table could require updating multiple tuples, increasing the risk of inconsistencies.

3. Informal Guidelines for Normalization

3.1 Avoid Redundant Data

- Redundancy increases storage needs and can lead to inconsistencies.
- Example: Storing a supplier's name and address in multiple tables instead of a single `SUPPLIER` table.

3.2 Use Meaningful Relations

- Tables should represent a single entity or relationship clearly.
- Example: Instead of having `MYRELATION123 (SUPNR, SUPNAME, SUPTWITTER, PRODNR, PRODNAME, ...)`, use two distinct relations:
 - `SUPPLIER(SUPNR, SUPNAME, SUPTWITTER,...)`
 - `PRODUCT(PRODNR, PRODNAME,...)`

3.3 Minimize NULL Values

- Excessive NULL values indicate poor table design.
- Example: A supplier's Twitter handle should not be part of `SUPPLIER` if many suppliers do not have one. Instead, a separate table `SUPPLIER_TWITTER (SUPNR, SUPTWITTER)` can be used.

4. Functional Dependencies & Prime Attributes

4.1 Functional Dependencies (FDs)

- A **functional dependency** (FD) exists when one attribute uniquely determines another.
 - X determines the value of Y.
- Example:
 - `SSN → ENAME` (A Social Security Number uniquely determines an employee's name.)
 - `{SSN, PNUMBER} → HOURS` (An employee's work hours depend on both SSN and Project Number.)

- Full dependencies:

(ProductName, UserName) → Rating
is a **full** functional dependency. Rating depends on ProductName **and** UserName.

- Partial dependencies:

(ProductName, UserName) → Password
is **partial** functional dependency because Password depends on UserName only. ProductName can be removed, and the dependency still holds.

4.2 Prime Attributes

- A **prime attribute type** is an attribute type that is part of a candidate key.
- Attribute types that are not part of any candidate key are **non-prime**.
- Example: In `R1(SSN, PNUMBER, PNAME, HOURS)`, the key `{SSN, PNUMBER}` makes both `SSN` and `PNUMBER` prime attributes, while `PNAME` and `HOURS` are non-prime.

5. Normal Forms in Normalization

5.1 First Normal Form (1NF) - Eliminating Multi-Valued & Composite Attributes

- **Condition:** All attributes must be atomic (no multi-valued or composite attributes).
- **Example:**
 - Unnormalized: `SUPPLIER(SUPNR, NAME(FIRST_NAME, LAST_NAME), SUPSTATUS)`
 - 1NF: `SUPPLIER(SUPNR, FIRST_NAME, LAST_NAME, SUPSTATUS)`

5.2 Second Normal Form (2NF) - Eliminating Partial Dependencies

- **Condition:** Must be in 1NF, and all non-key attributes must be fully functionally dependent on the entire primary key.
- **Example:**
 - `R1(SSN, PNUMBER, PNAME, HOURS)`
 - Issue: `PNAME` only depends on `PNUMBER`, not `SSN`.
 - Solution: Split into two tables:
 - `WORKS_ON(SSN, PNUMBER, HOURS)`
 - `PROJECT(PNUMBER, PNAME)`

5.3 Third Normal Form (3NF) - Eliminating Transitive Dependencies

- **Condition:** Must be in 2NF, and non-key attributes should not be transitively dependent on the primary key.
- **Example:**
 - `R1(SSN, ENAME, DNUMBER, DNAME, DMGRSSN)`
 - Issue: `DNAME` depends on `DNUMBER`, which depends on `SSN`.
 - Solution: Split into:
 - `EMPLOYEE(SSN, ENAME, DNUMBER)`
 - `DEPARTMENT(DNUMBER, DNAME, DMGRSSN)`

5.4 Boyce-Codd Normal Form (BCNF) - Eliminating Anomalies Beyond 3NF

- **Condition:** A relation is in BCNF if, for every functional dependency $X \rightarrow Y$, X is a superkey.
- **Example:**
 - `R1(SUPNR, SUPNAME, PRODNR, QUANTITY)`
 - Issue: `SUPNR → SUPNAME`, but `SUPNR` is not a superkey.

- Solution: Split into:

- `SUPPLIER(SUPNR, SUPNAME)`
- `SUPPLIES(SUPNR, PRODNR, QUANTITY)`

5.5 Fourth Normal Form (4NF) - Eliminating Multi-Valued Dependencies

- **Condition:** Must be in BCNF, and multi-valued dependencies should not exist unless the determinant is a superkey.
- **Example:**
 - `COURSE (course, instructor, textbook)`
 - Issue: A course can have multiple instructors and multiple textbooks, leading to redundancy.
 - Solution: Split into:
 - `COURSE_TEXTBOOK(course, textbook)`
 - `COURSE_INSTRUCTOR(course, instructor)`

6. Summary of Normalization Steps

Normal Form	Condition	Key Concept
1NF	Atomic values only	No multi-valued/composite attributes
2NF	1NF + No partial dependencies	Full dependency on primary key
3NF	2NF + No transitive dependencies	Every non-key attribute depends only on primary key
BCNF	3NF + Each determinant is a superkey	Stronger version of 3NF
4NF	BCNF + No multi-valued dependencies	Eliminates redundancy in multi-valued attributes

7. Advantages of Normalization

- ✓ **Better Data Integrity** – Avoids inconsistencies in data updates.

- ✓ **Efficient Storage** – Reduces redundant data.
- ✓ **Easier Querying** – Simplifies SQL queries.
- ✓ **Improved Flexibility** – Allows modifications without affecting large parts of the database.

8. When to Stop Normalization?

- While highly normalized databases avoid redundancy, they can introduce **performance issues** due to excessive joins.
- Sometimes, **denormalization** (adding redundancy intentionally) is used for **read-heavy** applications.
- A balance between **data integrity** and **query performance** must be maintained.

Exercises